

Resolución de la Actividad 1: Clase 01

Grupo 4

Integrantes:

- Avila, Aldana, legajo 1193721, alavila@uade.edu.ar
- Caivano, Alexander Francisco, legajo 1209389, acaivano@uade.edu.ar
- Casareski, Juan Ignacio, legajo 1176109, jcasareski@uade.edu.ar
- Segura, Juan Ignacio, legajo 1242159, jsegura@uade.edu.ar

1. Tabla de tipos de datos (Parte A)

Tipo de dato	Ejemplo en FlowOps	¿Estructurado, semi o no estructurado?	¿Cambia según tenant?	¿Crece mucho?	Observación
Tenant	Empresa cliente, configuración visual, tema, logo	Estructurado	Sí (poco)	Bajo	Datos relativamente estables
Proceso	Nodos, conexiones, reglas de decisión, formularios	Semi-estructurado	Sí	Medio	Puede variar mucho según el cliente
Instancia	Ejecución concreta de un proceso	Semi-estructurado	Sí	Alto	Se genera con cada ejecución
Evento	Paso ejecutado, error, decisión tomada, notificación	Semi-estructurado	Sí	Muy alto	Base de auditoría e historial
Tarea humana	Aprobación pendiente, datos de la tarea	Estructurado / semi-estructurado	Sí	Medio	Depende del proceso
Formulario	Campos dinámicos definidos por el tenant	Semi-estructurado / no estructurado	Sí	Medio	Estructura variable por tenant y proceso
Métricas	Tiempos de ejecución, duración por nodo	Estructurado	Sí	Alto	Necesita análisis analítico

Respuestas a las preguntas guía:

1. Datos a almacenar: tenants, definiciones de procesos, instancias en ejecución, eventos de auditoría, tareas humanas, formularios dinámicos, métricas y (opcionalmente) usuarios y roles.
2. Estructurados: tenants y métricas simples.
3. Semi-estructurados: procesos, instancias, eventos y tareas.
4. Datos que cambian según cliente o proceso: definiciones de procesos, formularios, reglas de decisión y flujos.
5. Datos que crecen más rápido: eventos de auditoría e instancias (escritura intensiva).
6. Datos que requieren baja latencia: estado actual de una instancia y tareas pendientes.
7. Datos que deben conservarse como historial: todos los eventos de ejecución (auditoría e histórico completo).

2. Comparación: solución relacional vs solución NoSQL (Parte B)

Problemas concretos de una implementación 100% relacional:

1. Formularios y procesos variables por tenant: las columnas fijas no se adaptan a campos dinámicos definidos por cada cliente.
2. Definiciones de flujos complejos: representar nodos y conexiones requiere múltiples tablas y JOINS, o campos genéricos (patrón EAV) muy poco eficientes.
3. Cambios frecuentes en el diseño del proceso: cualquier modificación de flujo obligaría a migraciones de esquema (ALTER TABLE) que afectan a todas las instancias.
4. Historial de eventos masivo: millones de filas con alto volumen de escrituras, con problemas de rendimiento y de escalabilidad vertical.
5. Consulta del estado actual de instancias: requiere muchos JOINS entre tablas de procesos, instancias, tareas y eventos, con latencia alta.
6. Tenants con estructuras distintas: es difícil mantener integridad referencial y rendimiento cuando cada tenant tiene flujos y formularios diferentes.
7. Escalabilidad: la base relacional se convierte en cuello de botella ante el crecimiento de tenants, instancias y eventos.

Problema	Por qué aparece en el modelo relacional	Qué característica NoSQL podría ayudar
Formularios variables por tenant	Las columnas fijas no se adaptan bien	Esquema flexible (documental)
Historial masivo de eventos	Muchas filas y alto volumen de escritura	Escalabilidad horizontal y escritura masiva
Procesos con nodos y conexiones	Estructura difícil de normalizar	Documentos o grafos
Estado actual de instancias	Necesidad de lectura rápida y actualizaciones frecuentes	Clave-valor (baja latencia)
Métricas de operación	Grandes volúmenes analíticos	Columnar o multidimensional / series temporales
Tenants con estructuras diferentes	Esquema rígido global	Esquema flexible por documento

3. Primer mapa de modelos NoSQL (Parte C)

Subsistema de FlowOps	Modelo NoSQL candidato	Justificación inicial
Definición de procesos	Documental	Un proceso es un documento con nodos, conexiones, reglas y formularios variables (JSON natural)
Estado actual de una instancia	Clave-valor	Se consulta frecuentemente por <code>instance_id</code> (lectura y actualización muy rápidas)
Historial de eventos / auditoría	Columnar / tabular	Escritura masiva y consultas por tenant, instancia y rango de fechas
Relaciones entre nodos del flujo	Grafo (opcional)	El flujo puede representarse directamente como nodos y relaciones (edges)
Métricas de tiempos de ejecución	Multidimensional / series temporales	Análisis de duración, demoras y cuellos de botella

Subsistema de FlowOps	Modelo NoSQL candidato	Justificación inicial
Formularios dinámicos y tareas	Documental	Estructura variable que se almacena junto con la instancia o la definición

4. Conclusión

FlowOps es un caso donde el modelo relacional tradicional muestra sus limitaciones principales: rigidez de esquema, dificultad para modelar estructuras jerárquicas y relaciones complejas (nodos y edges), y problemas de escalabilidad ante el volumen y la velocidad de escritura de eventos e instancias.

La flexibilidad de esquema de los modelos NoSQL permite que cada tenant defina sus propios procesos y formularios sin migraciones costosas. La escalabilidad horizontal y la escritura masiva de los motores NoSQL resuelven el crecimiento del historial de eventos. La baja latencia de clave-valor sirve para consultar el estado actual de instancias en tiempo real.

La variedad de datos (documentos estructurados, grafos de flujos, series temporales de métricas) justifica la persistencia políglota.

NoSQL no elimina la necesidad de diseñar correctamente el modelo de datos ni garantiza consistencia fuerte por defecto. Hay que analizar el teorema CAP y los patrones de acceso de cada subsistema.

Esta primera decisión técnica fija el punto de partida del TPO y muestra que FlowOps requiere las fortalezas que ofrecen los modelos NoSQL vistos en la materia.